

LabReSiD26 – Hands-On 15

Davide Ferrara – 518629

1 Esercizio 1: Buffer circolare a singolo thread

Cosa succede se `buf_get` viene chiamata su un buffer vuoto?

Né `buf_get` né `buf_put` controllano `count` prima di operare: il campo è un semplice contatore senza guardia. Chiamare `buf_get` su un buffer vuoto non produce nessun crash perché `head` rimane sempre dentro i limiti dell'array grazie al modulo N ; `head` avanza circolarmente e torna a 0, leggendo i valori stantii lasciati dal round precedente. Nel frattempo `count` scende a valori negativi senza alcun segnale di errore.

```
1 Buffer: 12 13 14 15
2 Tutti gli elementi prelevati correttamente.
3 Buffer: 12 13 14 15
4 Elemento extra: 12
5 Elemento extra: 13
6 Elemento extra: 14
7 Elemento extra: 15
8 Elemento extra: 12
9 Buffer: 12 13 14 15
```

Listing 1: Cinque `buf_get` extra su buffer vuoto

Le prime quattro chiamate restituiscono 12, 13, 14, 15: i valori dell'ultimo round ancora presenti nelle celle. La quinta restituisce 12 di nuovo perché `head` ha completato il giro circolare e riparte da 0.

In un'implementazione corretta `buf_get` dovrebbe controllare `count` prima di leggere: se `count = 0` tutti gli elementi sono già stati prelevati e l'operazione non è consentita. In questa versione `count` non viene mai controllato da nessuna funzione, quindi il buffer non ha alcuna protezione contro letture oltre il limite.

2 Esercizio 2: Due thread senza sincronizzazione

```
1 ==21199== Possible data race during write of size 4 at 0
   x1FFEFFE4E8 by thread #2
2 ==21199== Locks held: none
3 ==21199==    at 0x40011D6: buf_put (in /home/dff/git/
   reti/ho15/step2)
4 ==21199==    by 0x4001268: produttore (in /home/dff/git/
   reti/ho15/step2)
5 ==21199==    by 0x485F47B: mythread_wrapper (
   hg_intercepts.c:410)
6 ==21199==    by 0x49381B8: start_thread (pthread_create.
   c:454)
7 ==21199==    by 0x49BD043: clone (clone.S:100)
```

Listing 2: Output Helgrind – race condition in buf_put